HTML (1)

Basic structure of an HTML document

```
<!DOCTYPE html>
<html>

<head>
  <title>My document</title>
  <meta charset="utf-8">
</head>

<body>
  <p>Hello World!</p>
</body>

</html>
```

Simple table

```
<table>
  <tr>
    <td>Jill</td>
    <td>Smith</td>
  </tr>
  <tr>
    <td>John</td>
    <td>Doe</td>
  </tr>
</table>
```

Jill	Smith
John	Doe

Table with header and body Content

```
<table>
  <thead>
    <tr>
      <th>Vorname</th>
      <th>Nachname</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Jill</td>
      <td>Smith</td>
    </tr>
    <tr>
      <td>John</td>
      <td>Doe</td>
    </tr>
  </tbody>
</table>
```

Vorname	Nachname
Jill	Smith
John	Doe

Core Attributes

class	Class of an element	`<span class="important">My important text</span>`
id	Identification of an element	`<input type="text" id="first">`
style	CSS style for this element only	`<button style="background-color: green; border: none;">`

Hyperlinks / Directory Structure

```
<!-- link to a document on another server (absolute path) -->
<a href="http://example.com/index.html">Example</a>

<!-- link to a document in the same directory -->
<p>Use the <a href="contact.html">contact page</a> to ...</p>
<p>Use the <a href="./contact.html">contact page</a> to ...</p>

<!-- relative path to a subdirectory -->
<p>See <a href="pages/details.html">details</a></p>
<p>See <a href="./pages/details.html">details</a></p>

<!-- relative path (one level up from the current directory) -->
<p>Change to <a href="../en/index.html">english</a> language</p>

<!-- absolute path (starts at web root) -->
<p>Back to the <a href="/index.html">main page</a></p>
```

HTML (2)

Inline Elements

Element	Description	Example
<a>	Hyperlink	Example
	Bold text	This is bold text
 	Line break	This text contains a line break.
<button>	Clickable button	<button type="button">Click Me!</button>
<code>	Defines text as computer code	The HTML <code>button</code> tag defines a clickable button.
	Italic text	This is italic text
<img>	Image	<img src="smiley.jpg">
<input>	Input field	<input type="text" id="first" value="John">
	Generic inline element	My mother has blue eyes.
<textarea>	Multi-line text input	<textarea>A long text...</textarea>

Block Elements

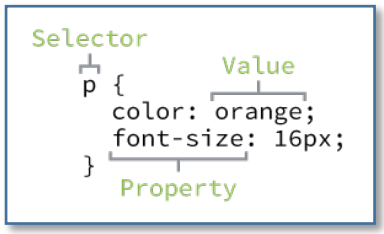
Element	Description	Example
	Generic block element	...
<form>	Form	<form>...</form>
<h1>-<h6>	Heading 1 - 6	This is a heading
	List item	Coffee
	Ordered list	 - Coffee - Tea
	Paragraph	This is a paragraph
<table>	Table	<table>...</table>
	Unordered list	 - Coffee - Tea

CSS

Inline style

```
<button style="border: none; color: white;">
  Click me
</button>
```

CSS Rule



Link to stylesheet

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

Selectors

.class	.button	Selects all elements with class="button"
#id	#button-1	Selects the element with id="button-1"
element	p	Selects all <p> elements
element, element	div, p	Selects all elements and all <p> elements
element element	div p	Selects all <p> elements inside elements
element > element	div > p	Selects all <p> elements where the parent is a element
*	*	Selects all elements

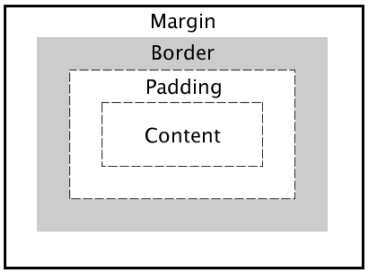
Properties for text and color

Declaration (example)	Description	Possible values (incomplete list)
font-weight: bold;	Bold text	normal | bold | lighter
font-style: italic;	Italic text	normal | italic | oblique
font-size: 250%;	250% font size.	16px | 50% | medium | small
text-align: center;	Horizontally align text to center.	left | right | center
text-transform: uppercase;	Show text in uppercase letters.	capitalize | uppercase | lowercase
color: rgb(0, 255, 0);	Set the text color to green.	green | #00ff00 | rgb(0,255,0)
background-color: blue;	Set the background color to blue. (Margin will remain transparent.)	red | #ff0000 | rgb(255,0,0)

Properties for spacing, sizes and borders

margin: 50px;	50 pixels margin in all directions.
margin-left: 1px;	1 pixel margin on the left.
padding: 30px;	30 Pixels padding in all directions.
padding-bottom: 2px;	2 pixels padding on the bottom.
border: 5px solid blue;	Solid blue border with 5 pixels width.
border-color: black;	Black border.
border-radius: 5px;	Round corners with a radius of 5 pixels.
height: 30px;	Set the height of the element to 30 pixels.
width: 50px;	Set the width of the element to 50 pixels.

CSS Box Model



Bootstrap / Forms

Grid

1	2	3

```
<div class="container">
  <div class="row">
    <div class="col-4">1</div>
    <div class="col-2">2</div>
    <div class="col-6">3</div>
  </div>
</div>
```

Form

```
<form>
  <div class="mb-3">
    <!-- Group of label and controls -->
  </div>
  <div class="mb-3">
    <!-- Group of label and controls -->
  </div>
  <button type="button" class="btn btn-primary">
    Submit
  </button>
</form>
```

Group of label and input element

```
<label for="name" class="form-label">Name</label>
<input type="text" class="form-control" id="name" placeholder="Enter name">
```

Name
Enter name

Group of label and text area

```
<label for="comment" class="form-label">Comment</label>
<textarea class="form-control" id="comment" rows="3"></textarea>
```

Comment

Group of label and drop-down list

```
<label for="lang" class="form-label">Language</label>
<select id="lang" class="form-select">
  <option selected>English</option>
  <option>German</option>
</select>
```

Language
English
English
German

Grid breakpoints

	Extra Small	Small	Medium	Large	Extra large
Dimensions	< 576px	≥ 576px	≥ 768px	≥ 992px	≥ 1200px
Class Prefix	col-	col-sm-	col-md-	col-lg-	col-xl-

HTML input types

checkbox	Defines a checkbox
color	Defines a color picker
date	Defines a date control (year, month, day)
email	Defines a field for an e-mail address
number	Defines a field for entering a number
password	Defines a password field
radio	Defines a radio button
text	Default. Defines a single-line text field
time	Defines a control for entering a time

Bootstrap classes

Table

```
<table class="table">...</table>
```

Button

```
<button type="button"
        class="btn btn-primary">
```

Responsive image
(image scales with the parent element)

```
<img src="..." class="img-fluid">
```

JavaScript

Data types

Boolean	true or false	true
Null	Intentional absence of a value.	null
Undefined	Variables with no value	undefined
Number	Number with or without decimals	4.5
String	Text	"Hello"
Object	Container for properties	{key: "value"}

Arithmetic operators

+	Addition	/	Division
-	Subtraction	%	Modulo
*	Multiplication		

Arrays

```
let items = ["A", "B", "C"];

// append an element
items.push("D");

for (let i=0; i<items.length; i++) {
  console.log(items[i]);
}
```

Include JavaScript files in HTML

```
<script src="script.js"></script>
```

Functions

```
function nameOfFunction(arg1, arg2) {
  return arg1 + arg2;
}
let result = nameOfFunction(arg1, arg2);
```

Conditional statements

```
if (condition1) {
    ...
} else if (condition2) {
    ...
} else {
    ...
}
```

Arrow Functions

```
(argument1, argument2) => { return argument1 + argument2; }
```

Loops

```
while (condition) {
  ...
}
```

```
for (initial value; condition; update) {
  ...
}
```

```
for (let element of elements) {
  ...
}
```

Comparison operators and logical operators

<	Less	===	Strict equality
<=	Less or equal	!==	Strict non-equality
>	Greater	&&	AND
>=	Greater or equal	||	OR
==	Equality	!	NOT

Variables

```
let name;
name = "Leia Organa";
let occupation = "Senator";
```

Objects

```
let person = {
  firstName: "Luke",
  lastName: "Skywalker"
};

person.firstName = "Anakin";
let lName = person["lastName"];

for (let key in person) {
  console.log(key);
}
```

Nested Objects

```
let planet = {
  name: "Mars",
  surfaceTemperatureCelsius: {
    min: -143,
    max: 35
  },
  satellites: [
    { name: "Phobos", discoveryDate: "18 August 1877"},
    { name: "Deimos", discoveryDate: "12 August 1877"}
  ]
}
```

Svelte - Bindings and Blocks

Minimal page

```
<script>
  let message = $state("Hello World");
</script>

<div>
  <h1>{message}</h1>
</div>

<style>
  h1 {
    color: #ff3e00;
    text-transform: uppercase;
  }
</style>
```

Variables as attribute values

```
<script>
  let image = $state("/img/Altura.png");
</script>

<img src={image}>
```

Value bindings

```
<script>
  let input = $state("");
</script>

<input type="text" bind:value={input}>
```

Bind an HTML event to a function

```
<script>
  let name = "Luke";

  function sayHello() {
    alert("Hello, I'm " + name);
  }
</script>

<button onclick={sayHello}>Click me</button>
```

Each blocks

- Get a lightsaber
- Use the force

```
<script>
  let todos = $state([
    "Get a lightsaber",
    "Use the force"
  ]);
</script>

<div>
  <ul>
    {#each todos as todo}
      <li>{todo}</li>
    {/each}
  </ul>
</div>
```

If – Else if - Else blocks

```
<script>
  let x = $state(7);
</script>

{#if x > 10}
    <p>Above 10</p>
{:else}
  {#if x < 5}
    <p>Below 5</p>
  {:else if x === 5}
    <p>Exactly 5</p>
  {:else}
    <p>Above 5, up to 10</p>
  {/if}
{/if}
```

Svelte - Components

Importing and using components

*src/routes/**+page.svelte***

```
<script>
  import ComponentA from "#lib/components/ComponentA.svelte";
  import ComponentB from "#lib/components/ComponentB.svelte";
</script>

<p>Let's add some components!</p>

<ComponentA/>
<ComponentB/>
<ComponentB/>
```

Let's add some components!

This is Component A

This is Component B

This is Component B

*src/lib/components/**ComponentA.svelte***

```
<p>This is Component A</p>

<style>
    p {
        color: red;
    }
</style>
```

*src/lib/components/**ComponentB.svelte***

```
<p>This is Component B</p>

<style>
    p {
        color: blue;
        font-style: italic;
        font-weight: bold;
    }
</style>
```

Passing *static* data to components with props

*src/routes/**+page.svelte***

```
<script>
    import MessageBox from "#lib/components/MessageBox.svelte";
</script>

<MessageBox title="Welcome" message="Hello World!" />
```

*src/lib/components/**MessageBox.svelte***

```
<script>
    let { title, message = "none" } = $props();
</script>

<h1>{title}</h1>
<p>Your message: {message}</p>
```

Passing *objects* as props

*src/routes/**+page.svelte***

```
<script>
    import ChatMessage from "#lib/components/ChatMessage.svelte";
    let messages = $state([
      {
        text: "Hello PT",
        sender: "Max"
      },
      {
        text: "Hello DM",
        sender: "Alex"
      }
    ])
</script>

{#each messages as message}
  <ChatMessage message={message}/>
{/each}
```

Chat Message

Max: **Hello PT**

Chat Message

Alex: **Hello DM**

*src/lib/components/**ChatMessage.svelte***

```
<script>
  let { message } = $props();
</script>

<div class="box">
  <h2>Chat Message</h2>
  <p>{message.sender}: <b>{message.text}</b></p>
</div>
```

Svelte – Projects, Routing, Loading Data

Create an app with SvelteKit / Install dependencies / Run development server

Terminal

```
npx sv create my-app
cd my-app
npm install
npm run dev
```



Routing

```
<a href="/artists/12345">Beatles</a>
```

```
<a href={"/artists/" + artist._id}>{artist.name}</a>
```

```
<a href="/artists">Artists</a>
```

Layout

routes/+layout.svelte

```
<script>
  import './styles.css';
  let { children } = $props();
</script>

<div class="container">
  <nav>
    <a href="/">home</a>
    <a href="/contact">contact</a>
    <a href="/users">users</a>
  </nav>

  {@render children()}
</div>
```

load function

artists/+page.server.js

```
export async function load() {
  return {
    artists: ['Daft Punk', 'Chilly Gonzalez']
  }
}
```

artists/+page.svelte

```
<script>
  let { data } = $props();
</script>

<ul>
  {#each data.artists as artist}
    <li>{artist}</li>
  {/each}
</ul>
```

- Daft Punk
- Chilly Gonzalez

load function with path parameters and a library function

```
http://localhost:5173/artists/12345
```

/artists/[artist_id]/+page.server.js

```
import db from '#lib/server/db.js';

export async function load( { params } ) {

  return {
    artist: await db.getArtist( params.artist_id )
  };
}
```



Svelte – Forms

Form Actions

*artists/create/**+page.svelte***

```
<form method="POST" action="?/create">
  <div class="mb-3">
    <label for="name" class="form-label">Name</label>
    <input name="name" id="name" class="form-control" type="text" />
  </div>
  <div class="mb-3">
    <label for="origin" class="form-label">Origin</label>
    <input name="origin" id="origin" class="form-control" type="text" />
  </div>
  <button type="submit" class="btn btn-primary">Add Artist</button>
</form>
```

*artists/create/**+page.server.js***

```
import db from '#lib/server/db.js';

export const actions = {
  create: async ({request}) => {
    const data = await request.formData();
    let artist = {
      name: data.get("name"),
      origin: data.get("origin"),
    }
    await db.createArtist(artist);
  }
}
```

Form Feedback

*artists/create/**+page.svelte***

```
<script>
  let { form } = $props();
</script>

<form method="POST" action="?/create">
<!-- ... -->
</form>

{#if form?.success}
  <p>Artist created</p>
{/if}
```

*artists/create/**+page.server.js***

```
import db from '#lib/server/db.js';

export const actions = {
  create: async ({request}) => {
    // ...
    return { success: true }
  }
}
```

Svelte – MongoDB and Environment Variables

Environment Variables

src/env.js

```
import { defineEnvVars } from
'@sveltejs/kit/env';

export const variables = defineEnvVars({
    DB_URI: {}
});
```

Declare environment variables in src/env.js.

Store their values in .env during development. Variables are private by default.

.env

```
DB_URI=mongodb+srv://...
```

The values of environment variables are stored in the file *.env*

MongoDB + Svelte

*lib/server/**db.js***

```
import { MongoClient, ObjectId } from 'mongodb';
import { DB_URI } from '$app/env/private';

const client = new MongoClient(DB_URI);
await client.connect();
const db = client.db('MusicDB') // select database

async function getArtists() {
    let artists = [];
    try {
        const collection = db.collection('artists');
        const query = {};
        artists = await collection.find(query).toArray();
        artists.forEach(artist => {
            artist._id = artist._id.toString(); // convert ObjectId to String
        });
    } catch (error) {
        // errorhandling
    }
    return artists;
}

// export functions so that they can be used in other files
export default { getArtists }
```

*routes/artists/**+page.server.js***

```
import db from '#lib/server/db.js'

export async function load() {

    return {
        artists: await db.getArtists()
    }
}
```